LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 4&5

THE CORE OF OOP & ENCAPSULATION

Disusun Oleh:

ELISABETH NAINGGOLAN

105224037

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. PENDAHULUAN

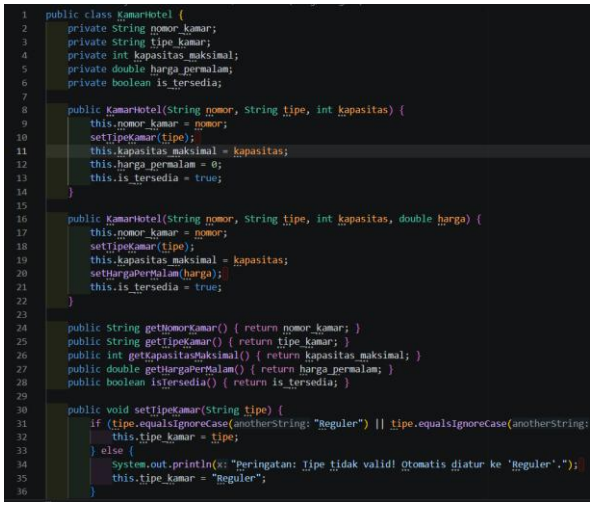
Praktikum ini bertujuan untuk mengimplementasikan konsep dasar Pemrograman Berorientasi Objek (PBO) dalam bahasa Java. Fokus utama praktikum kali ini adalah pembuatan *class* dan *object*, penggunaan *constructor*, penerapan *encapsulation* (private variables dan setter/getter), serta teknik *method overloading* untuk menciptakan fungsi yang lebih fleksibel dengan nama yang sama namun parameter berbeda.

II. VARIABEL

no	Nama variabel	Jenis / lingkup	Tipe data	Fungsi
1.	nomor_kamar	Atribut (Private)	string	Menyimpan nomor identitas kamar secara permanen.
2.	tipe_kamar	Atribut (Private)	string	Menyimpan kategori kamar (Reguler/Premium/Suite).
3.	kapasitas_maksimal	Atribut (Private)	Int	Menyimpan batas daya tampung maksimal tamu.
4.	harga_permalam	Atribut (Private)	Double	Menyimpan tarif sewa per malam (min. 50.000).
5.	is_tersedia	Atribut (Private)	Boolean	Menyimpan status ketersediaan (true/false).
6.	Nomor	Parameter	String	Input nomor pada *constructor*.
7.	Type	Parameter	String	Input tipe pada *constructor* dan *setter*.
8.	Kapasitas	Parameter	Int	Input kapasitas pada *constructor*.
9.	Harga	Parameter	Double	Input harga pada *constructor* dan *setter*.
10.	jumlahTamu	Parameter	Int	Input tamu pada method pesanKamar (Overloading).
11.	jumlahMalam	Parameter	Int	Input durasi pada method hitungTotalBayar.
12.	kodeVoucher	Parameter	String	Input kode promo pada method hitungTotalBayar.
13.	Total	Variabel Lokal	Double	Menampung hasil perhitungan harga sebelum diskon.
14.	Kamar1	Objek (Local)	kamarHotel	Instansiasi objek pertama di method main.
15.	Kamar2	Objek (Local)	kamarHotel	Instansiasi objek kedua di method main.

16.	Tagihan1	Variable lokal	Double	Menyimpan hasil return biaya untuk kamar1.
17.	Tagihan2	Variabel Lokal	Double	Menyimpan hasil return biaya untuk kamar2.
18.	Args	Parameter	String[]	Parameter utama pada method main.
19.	This	Keyword	kamarHotel	Referensi internal yang menunjuk ke objek yang sedang berjalan (digunakan di constructor/setter).
20.	0.8	Literal	Double	Faktor pengali untuk diskon 20% (secara teknis diperlakukan sebagai nilai dalam logika).
21.	50000	Literal	double	Nilai batas minimum harga dalam validasi setHargaPerMalam.

III. DOKUMENTASI DAN PEMBAHASAN KODE



```java
public class KamarHotel {
    private String nomor_kamar;
    private String tipe_kamar;
    private int kapasitas_maksimal;
    private double harga_permalam;
    private boolean is_tersedia;

    public KamarHotel(String nomor, String tipe, int kapasitas) {
        this.nomor_kamar = nomor;
        setTipeKamar(tipe);
        this.kapasitas_maksimal = kapasitas;
        this.harga_permalam = 0;
        this.is_tersedia = true;
    }

    public KamarHotel(String nomor, String tipe, int kapasitas, double harga) {
        this.nomor_kamar = nomor;
        setTipeKamar(tipe);
        this.kapasitas_maksimal = kapasitas;
        setHargaPerMalam(harga);
        this.is_tersedia = true;
    }

    public String getNomorKamar() { return nomor_kamar; }
    public String getTipeKamar() { return tipe_kamar; }
    public int getKapasitasMaksimal() { return kapasitas_maksimal; }
    public double getHargaPerMalam() { return harga_permalam; }
    public boolean isTersedia() { return is_tersedia; }

    public void setTipeKamar(String tipe) {
        if (tipe.equalsIgnoreCase(anotherString: "Reguler") || tipe.equalsIgnoreCase(anotherString:
            this.tipe_kamar = tipe;
        } else {
            System.out.println(x: "Peringatan: Tipe tidak valid! Otomatis diatur ke 'Reguler'.");
            this.tipe_kamar = "Reguler";
        }
```

```java
    public void setHargaPerMalam(double harga) {
        if (harga < 50000) {
            System.out.println(x: "Peringatan: Harga di bawah minimum! Otomatis diatur ke 50000.");
            this.harga_permalam = 50000;
        } else {
            this.harga_permalam = harga;
        }
    }

    public void pesanKamar() {
        if (is_tersedia) {
            is_tersedia = false;
            System.out.println("Pemesanan berhasil untuk Kamar " + nomor_kamar);
        } else {
            System.out.println("Peringatan: Kamar " + nomor_kamar + " sudah terisi!");
        }
    }

    public void pesanKamar(int jumlahTamu) {
        if (!is_tersedia) {
            System.out.println("Peringatan: Kamar " + nomor_kamar + " sudah terisi!");
        } else if (jumlahTamu > kapasitas_maksimal) {
            System.out.println("Pesanan Ditolak: Jumlah tamu (" + jumlahTamu + ") melebihi kapasitas!");
        } else {
            is_tersedia = false;
            System.out.println("Pemesanan berhasil untuk " + jumlahTamu + " tamu di Kamar " + nomor_kamar);
        }
    }

    public void batalPesan() {
        is_tersedia = true;
        System.out.println("Pemesanan Kamar " + nomor_kamar + " telah dibatalkan.");
    }

    public double hitungTotalBayar(int jumlahMalam) {
        return harga_permalam * jumlahMalam;
    }

    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
        double total = hitungTotalBayar(jumlahMalam);
        if (kodeVoucher.equalsIgnoreCase(anotherString: "PROMO") && jumlahMalam >= 3) {
            System.out.println(x: "Voucher Berhasil: Diskon 20% diterapkan.");
            return total * 0.8;
        } else {
            System.out.println(x: "Voucher Gagal: Syarat tidak terpenuhi atau kode salah. Menggunakan harga normal.");
            return total;
        }
    }

    public void tampilkanInformasi() {
        System.out.println(x: "=== Struk Informasi Kamar ===");
```

```java
public class MainTHT {
    Run | Debug
    public static void main(String[] args) {
        KamarHotel kamar1 = new KamarHotel(nomor: "101", tipe: "Presidential", kapasitas: 2, -10000);
        KamarHotel kamar2 = new KamarHotel(nomor: "202", tipe: "Suite", kapasitas: 2, harga: 200000);

        System.out.println(x: "\n--- Simulasi Kamar 1 ---");
        double tagihan1 = kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO");
        System.out.println("Total Tagihan Kamar 1: Rp " + tagihan1);

        System.out.println(x: "\n--- Simulasi Kamar 2 ---");
        kamar2.pesanKamar(jumlahTamu: 4);
        kamar2.pesanKamar(jumlahTamu: 2);
        kamar2.pesanKamar();

        double tagihan2 = kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO");
        System.out.println("Total Tagihan Kamar 2 (Diskon 20%): Rp " + tagihan2);

        System.out.println(x: "\n--- Hasil Akhir Kondisi Kamar ---");
        kamar1.tampilkanInformasi();
        kamar2.tampilkanInformasi();

    }
}
```

Pembahasan KamarHotel

```java
1  public class KamarHotel {
2      private String nomor_kamar;
3      private String tipe_kamar;
4      private int kapasitas_maksimal;
5      private double harga_permalam;
6      private boolean is_tersedia;
```

Bagian 1: Struktur Atribut dan Encapsulation Pada bagian awal, program mendefinisikan identitas dari objek kamar hotel.

- Baris 2-6: Mendeklarasikan variabel dengan akses modifier private. Ini adalah implementasi Encapsulation agar data tidak bisa diubah secara langsung dari luar class. Data yang disimpan meliputi nomor kamar, tipe, kapasitas, harga, dan status ketersediaan.

```java
8   public KamarHotel(String nomor, String tipe, int kapasitas) {
9       this.nomor_kamar = nomor;
10      setTipeKamar(tipe);
11      this.kapasitas_maksimal = kapasitas;
12      this.harga_permalam = 0;
13      this.is_tersedia = true;
14  }
15
16  public KamarHotel(String nomor, String tipe, int kapasitas, double harga) {
17      this.nomor_kamar = nomor;
18      setTipeKamar(tipe);
19      this.kapasitas_maksimal = kapasitas;
20      setHargaPerMalam(harga);
21      this.is_tersedia = true;
22  }
```

Bagian 2: Constructor Overloading Program menyediakan dua cara untuk membuat objek kamar, yang menunjukkan fleksibilitas dalam inisialisasi.

- (Baris 8-14): Digunakan jika harga belum diketahui. Status is_tersedia otomatis diset true dan tipe kamar divalidasi melalui method setTipeKamar.
- (Baris 16-22): Digunakan untuk pendaftaran kamar lengkap dengan harganya. Perbedaannya terletak pada penggunaan setHargaPerMalam(harga) untuk memastikan harga yang dimasukkan valid.

```
24  public String getNomorKamar() { return nomor_kamar; }
25  public String getTipeKamar() { return tipe_kamar; }
26  public int getKapasitasMaksimal() { return kapasitas_maksimal; }
27  public double getHargaPerMalam() { return harga_permalam; }
28  public boolean isTersedia() { return is_tersedia; }
29
30  public void setTipeKamar(String tipe) {
31      if (tipe.equalsIgnoreCase(anotherString: "Reguler") || tipe.equalsIgnoreCase(anotherString: "Premium") || tipe.equalsIgnoreCase(anotherString: "Suite")) {
32          this.tipe_kamar = tipe;
33      } else {
34          System.out.println(x: "Peringatan: Tipe tidak valid! Otomatis diatur ke 'Reguler'.");
35          this.tipe_kamar = "Reguler";
36      }
37  }
38
39  public void setHargaPerMalam(double harga) {
40      if (harga < 50000) {
41          System.out.println(x: "Peringatan: Harga di bawah minimum! Otomatis diatur ke 50000.");
42          this.harga_permalam = 50000;
43      } else {
44          this.harga_permalam = harga;
45      }
46  }
```

Bagian 3: Method Getter dan Setter (Validasi) Method ini berfungsi sebagai jembatan untuk mengakses dan mengubah data *private*.

- (Baris 24-28): Getter Fungsi pendukung untuk mengambil nilai dari atribut *private* (seperti nomor atau harga) saat dibutuhkan oleh class lain.
- (Baris 30-37): Method setTipeKamar Menggunakan struktur kontrol if-else untuk memastikan input hanya berisi "Reguler", "Premium", atau "Suite". Jika input di luar itu, program memberikan peringatan dan memberikan nilai *default*.
- (Baris 39-46): Method setHargaPerMalam Berfungsi sebagai filter keuangan. Jika harga di bawah 50.000, program akan memaksa nilai tersebut menjadi 50.000 untuk mencegah kesalahan input data.

```
47
48  public void pesanKamar() {
49      if (is_tersedia) {
50          is_tersedia = false;
51          System.out.println("Pemesanan berhasil untuk Kamar " + nomor_kamar);
52      } else {
53          System.out.println("Peringatan: Kamar " + nomor_kamar + " sudah terisi!");
54      }
55  }
56
57  public void pesanKamar(int jumlahTamu) {
58      if (!is_tersedia) {
59          System.out.println("Peringatan: Kamar " + nomor_kamar + " sudah terisi!");
60      } else if (jumlahTamu > kapasitas_maksimal) {
61          System.out.println("Pesanan Ditolak: Jumlah tamu (" + jumlahTamu + ") melebihi kapasitas!");
62      } else {
63          is_tersedia = false;
64          System.out.println("Pemesanan berhasil untuk " + jumlahTamu + " tamu di Kamar " + nomor_kamar);
65      }
66  }
67
68  public void batalPesan() {
69      is_tersedia = true;
70      System.out.println("Pemesanan Kamar " + nomor_kamar + " telah dibatalkan.");
71  }
```

Bagian 4: Logika Pemesanan (Method Overloading) Bagian ini mengatur interaksi pengguna dengan status ketersediaan kamar.

- (Baris 48-55): pesanKamar(), Logika sederhana untuk mengecek apakah kamar kosong. Jika kosong, status diubah menjadi false (dipesan).

- (Baris 57-66): pesanKamar(int jumlahTamu) Pengembangan dari method sebelumnya dengan tambahan parameter. Program akan membandingkan jumlahTamu dengan kapasitas_maksimal. Jika tamu terlalu banyak, pesanan otomatis ditolak.
- (Baris 68-71): batalPesan() Mengembalikan status is_tersedia menjadi true agar kamar bisa dipesan kembali oleh orang lain.

```
72
73      public double hitungTotalBayar(int jumlahMalam) {
74          return harga_permalam * jumlahMalam;
75      }
76
77      public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
78          double total = hitungTotalBayar(jumlahMalam);
79          if (kodeVoucher.equalsIgnoreCase(anotherString: "PROMO") && jumlahMalam >= 3) {
80              System.out.println(x: "Voucher Berhasil: Diskon 20% diterapkan.");
81              return total * 0.8;
82          } else {
83              System.out.println(x: "Voucher Gagal: Syarat tidak terpenuhi atau kode salah. Menggunakan harga normal.");
84              return total;
85          }
86      }
```

Bagian 5: Perhitungan Biaya dan Voucher Implementasi logika bisnis untuk menentukan nilai transaksi pelanggan.

- (Baris 73-75): hitungTotalBayar(int jumlahMalam, Perhitungan dasar (Harga x Durasi).
- (Baris 77-86): hitungTotalBayar(int jumlahMalam, String kodeVoucher) ,Logika promo. Jika pengguna memasukkan kode "PROMO" dan menginap minimal 3 malam, maka total bayar dikalikan 0.8 (Diskon 20%). Jika syarat tidak terpenuhi, harga kembali normal.

```
87
88      public void tampilkanInformasi() {
89          System.out.println(x: "=== Struk Informasi Kamar ===");
90          System.out.println("Nomor Kamar   : " + nomor_kamar);
91          System.out.println("Tipe Kamar    : " + tipe_kamar);
92          System.out.println("Kapasitas Max : " + kapasitas_maksimal);
93          System.out.println("Harga/Malam   : Rp " + harga_permalam);
94          System.out.println("Status        : " + (is_tersedia ? "Tersedia" : "Dipesan"));
95          System.out.println(x: "----------------------------");
96      }
97  }
```

Bagian 6: Output Informasi

- (Baris 88-96): Method tampilkanInformasi Berfungsi mencetak struk atau ringkasan detail kamar ke terminal. Di sini terdapat penggunaan operator *ternary* (is_tersedia ? "Tersedia" : "Dipesan") untuk mengubah nilai *boolean* menjadi teks yang mudah dibaca pengguna.

Pembahasan MainTHT

```
1  public class MainTHT {
2      public static void main(String[] args) {
3          KamarHotel kamar1 = new KamarHotel("101", "Presidential", 2, -10000);
4          KamarHotel kamar2 = new KamarHotel("202", "Suite", 2, 200000);
5
```

Bagian 1: Instansiasi Objek (Pembuatan Kamar) Pada tahap awal, program melakukan pembuatan dua objek berbeda menggunakan *constructor* yang telah didefinisikan sebelumnya.

- Baris 3-4: Program membuat dua objek, yaitu kamar1 dan kamar2.
    - kamar1 diinisialisasi dengan harga negatif (-10000) dan tipe "Presidential" untuk menguji fitur validasi. Karena "Presidential" tidak ada dalam daftar tipe yang valid, sistem otomatis mengubahnya menjadi "Reguler", dan harga negatif akan dikoreksi menjadi Rp 50.000 melalui method *setter*.
    - kamar2 diinisialisasi dengan data yang valid (Suite, harga 200.000).

```
6          System.out.println(x: "\n--- Simulasi Kamar 1 ---");
7          double tagihan1 = kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO");
8          System.out.println("Total Tagihan Kamar 1: Rp " + tagihan1);
9
```

Bagian 2: Simulasi Perhitungan Biaya (Kamar 1) Bagian ini menunjukkan bagaimana method *overloading* untuk perhitungan biaya bekerja.

- Baris 7-8: Program memanggil hitungTotalBayar(2, "PROMO") untuk kamar1. Meskipun menggunakan kode "PROMO", diskon tidak akan diterapkan karena syarat minimal menginap adalah 3 malam, sedangkan pada parameter hanya diisi 2 malam. Hasilnya disimpan dalam variabel tagihan1.

```
10         System.out.println(x: "\n--- Simulasi Kamar 2 ---");
11         kamar2.pesanKamar(jumlahTamu: 4);
12         kamar2.pesanKamar(jumlahTamu: 2);
13         kamar2.pesanKamar();
```

Bagian 3: Simulasi Pemesanan dan Validasi Kapasitas (Kamar 2) Pada bagian ini, program menguji logika pemesanan kamar dengan berbagai skenario.

- Baris 11: kamar2.pesanKamar(4), Percobaan pemesanan untuk 4 tamu. Karena kapasitas kamar 2 hanya untuk 2 orang, pesanan ini akan ditolak oleh sistem.
- Baris 12: kamar2.pesanKamar(2), Percobaan pemesanan untuk 2 tamu. Karena sesuai kapasitas dan kamar masih tersedia, pemesanan berhasil dan status kamar berubah menjadi "Dipesan".
- Baris 13: kamar2.pesanKamar() — Percobaan pemesanan ulang pada kamar yang sama. Sistem akan memberikan peringatan bahwa kamar sudah terisi.

```
15         double tagihan2 = kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO");
16         System.out.println("Total Tagihan Kamar 2 (Diskon 20%): Rp " + tagihan2);
17
18         System.out.println(x: "\n--- Hasil Akhir Kondisi Kamar ---");
19         kamar1.tampilkanInformasi();
20         kamar2.tampilkanInformasi();
```

Bagian 4: Simulasi Diskon dan Output Akhir Bagian penutup untuk melihat hasil akhir dari seluruh proses manipulasi data.

- Baris 15-16: Program menghitung tagihan untuk kamar2 dengan durasi 4 malam dan kode "PROMO". Karena syarat terpenuhi (>= 3 malam), diskon 20% berhasil diterapkan.
- Baris 19-20: Memanggil method tampilkanInformasi() untuk kedua objek guna mencetak ringkasan status akhir kamar ke terminal, menunjukkan perubahan data setelah proses validasi dan pemesanan.

IV. KESIMPULAN

Secara keseluruhan, praktikum pada proyek KamarHotel ini memberikan pemahaman bahwa penggunaan konsep Pemrograman Berorientasi Objek (PBO) membuat struktur kode menjadi jauh lebih rapi dan aman.

Dengan menerapkan *Encapsulation*, data tidak bisa diubah sembarangan karena variabel dibuat private, sehingga setiap perubahan harus melewati validasi di method *setter* untuk mencegah kesalahan input seperti harga yang terlalu murah atau tipe kamar yang tidak terdaftar. Selain itu, fitur *Method Overloading* sangat membantu dalam memberikan fleksibilitas pada program; kita bisa membuat fungsi dengan nama yang sama namun kegunaan yang berbeda, misalnya untuk memproses pesanan dengan atau tanpa jumlah tamu.

Hal ini membuktikan bahwa PBO sangat efektif untuk mensimulasikan logika bisnis di dunia nyata, seperti sistem manajemen hotel, ke dalam bentuk program yang fungsional dan mudah untuk dikembangkan lagi.

V. DAFTAR PUSTAKA

Asisten Praktikum Pemrograman Berorientasi Objek (2026). *Modul Praktikum Pemrograman Berorientasi Objek - Modul 4 & 5: Encapsulation and Method Overloading*. Jakarta: Program Studi Ilmu Komputer, Universitas Pertamina.

Dosen Pengampu PBO. (2026). *Materi Kuliah PBO: Implementasi Getter-Setter dan Fleksibilitas Method Overloading*. Jakarta: Fakultas Sains dan Komputer, Universitas Pertamina.